Chapter 5: Using Controls

Controls are interactive objects that you can place on dialog boxes, toolbars, and other windows. Users interact with controls to manipulate and enter data. When you build an application with MFC, you can use the following types of controls.

- Windows common controls

  These controls are provided by the Windows operating system. The Microsoft Foundation Class (MFC) Library encapsulates these controls, so that they can be programmed and used in dialog boxes and other Windows controls.

- Internet Explorer 4.0 common controls

  These controls include **CComboBoxEx**, **CIPAddressCtrl**, **CMonthCalCtrl**, **CProgressCtrl**, and **CReBar**.

- ActiveX controls

  You can use ActiveX controls in applications for Windows or in HTML pages on the World Wide Web. These controls are reusable and must be loaded into a control container.

- Control classes supplied by the MFC Library

  These classes include **CBitmapButton**, **CCheckListBox**, and **CDragListBox**.

This chapter describes the procedure for adding controls to an MFC-based application.

Objectives

After completing this chapter, you will be able to:

- Add Windows common controls to MFC-based applications.
- Add Internet Explorer 4.0 common controls to MFC-based applications.
- Add ActiveX controls to MFC-based applications.
- Add controls supplied by the MFC Library to MFC-based applications.

128 Windows Common Controls

Windows common controls are provided by the Windows operating system. These controls are programmable and can be added to dialog boxes and other windows. MFC provides a number of classes to encapsulate Windows common controls.

In this section, you will learn to create Windows common controls, set control attributes, and process notification messages.

This section includes the following topics:

- MFC Classes for Windows Common Controls
- Creating Windows Common Controls
- Setting Windows Common Control Attributes
- Processing Notification Messages from Windows Common Controls

Notes

129 MFC Classes for Windows Common Controls

MFC provides a number of control classes that encapsulate Windows common controls. These controls include **CSliderCtrl**, **CSpinButtonCtrl**, **CListCtrl**, **CToolBarCtrl**, **CStatusBarCtrl**, **CAnimateCtrl**, **CHeaderCtrl**, **CHotKeyCtrl**, and **CRichEditCtrl**. The **CWnd** class is the base class for all these controls.

The following table provides a brief description of some Windows common controls.

Type of control	MFC control class	Description
Edit controls	**CRichEditCtrl**	A window in which the user can enter and edit text. The characters and paragraph in the control can be formatted, and the control can include embedded ActiveX objects.
Controls that represent numbers	**CSliderCtrl**	A window containing arrow buttons and a slider, which the user moves to select a value.
	CSpinButtonCtrl	A pair of arrow buttons that the user can click to increment or decrement a value.
Lists	**CListCtrl**	Displays a collection of items, each consisting of an icon and a label, in a manner similar to the right pane of the Windows 95 Explorer.
Toolbars and Status Bars	**CToolBarCtrl**	Provides the functionality of the Windows toolbar common control. Most MFC-based applications use **CToolBar** instead of this class.
	CStatusBarCtrl	A horizontal window usually divided into panes, in which an application can display status information. Most MFC-based applications use **CStatusBar** instead of this class.

Notes

Type of control	MFC control class	Description
Miscellaneous controls	**CAnimateCtrl**	Displays a simple video clip.
	CHotKeyCtrl	Displays a text representation of the key combination the user types into the hot key control. **CHotKeyCtrl** is often used in dialog boxes that prompt the user to assign a hot key. You must program the control to retrieve the values describing the hot key and associate the hot key with a window or thread.
	CHeaderCtrl	Displays titles or labels for columns.

For information about these controls, search for "Windows common controls" in MSDN Library Visual Studio 6.0.

Notes

130 Creating Windows Common Controls

MFC provides classes to wrap common controls. There are two ways you can create a common control in an MFC-based application:

- Use code to create the control by creating an instance of the MFC control class and then calling the object's **Create** function.
- Use the dialog editor to add the control to a dialog box.

▶ **To create a Windows common control object by using code**

1. Create an instance of the control in a dialog box or frame-window object.
2. Call the control's **Create** member function in the **OnInitDialog** function for dialog boxes and in the **OnCreate** function for frame windows.

The following example shows the declaration of a **CEdit** object and a **CSpinButtonCtrl** object:

```
class CSpinEditDlg : public CDialog
{
  protected:
    CEdit m_wndCEdit; // Embedded edit object
    CSpinButtonCtrl m_wndCSpinButton; // Embedded Spin Button object
  public:
    Virtual BOOL OnInitDialog();
};
```

Notes

To see sample code that creates an edit control and a spin button control, click this icon.
 (CD-ROM displays the following sample code.)

```
//Samples\CH05\SpinButton
//Creates a spin button and its buddy window
BOOL CSpinEditDlg::OnInitDialog()
{
    CDialog::OnInitDialog();
    CRect rect1(10, 30, 100, 55);
    CRect rect2(101, 30, 110, 55);
    m_wndCEdit.Create(WS_CHILD | WS_VISIBLE | WS_TABSTOP |
     ES_AUTOHSCROLL | WS_BORDER, rect1, this, IDC_BUDDY);
    m_wndCSpinButton.Create(WS_CHILD | WS_VISIBLE | WS_TABSTOP |
     ES_AUTOHSCROLL | WS_BORDER|UDS_SETBUDDYINT,
    rect2, this, IDC_SPIN_BUTTON);
    m_wndCSpinButton.SetFocus();
    m_wndCSpinButton.SetBuddy(&m_wndCEdit);
    m_wndCSpinButton.SetRange(0,100);
    m_pCStaticCm = (CStatic*)GetDlgItem(IDC_LENGTH_CM);
    m_pMph=(CProgressCtrl*)GetDlgItem(IDC_MPH);
    m_pKmph=(CProgressCtrl*)GetDlgItem(IDC_KMPH);
    m_pMph->SetRange(0,200);
    m_pKmph->SetRange(0,200);
    m_pMph->SetStep(1);
    m_pKmph->SetStep(1);
    SetDlgItemInt(IDC_BUDDY,m_wndCSpinButton.GetPos());
    return TRUE;
}
```

In the previous sample code, the **OnInitDialog** function sets up two rectangles. The first rectangle is for the edit control and the second is for the spin button control. Calling the **Create** function creates the edit control and the spin button control as shown in the following example:

```
m_wndCEdit.Create(WS_CHILD|WS_VISIBLE|WS_TABSTOP|ES_AUTOHSCROLL|WS_BORDER,
                    rect1, this, IDC_EDIT1);

m_wndCSpinButton.Create(WS_CHILD|WS_VISIBLE|WS_TABSTOP|ES_AUTOHSCROLL|WS_BORDE
R|UDS_SETBUDDYINT ,
 rect2, this, IDC_BUTTON1);
```

The style UDS_SETBUDDYINT causes the spin button control to set the text of the buddy window when the user clicks the spin button. A buddy window is an edit control that displays the value of the current position of the spin button control. You associate an edit control with a spin button control by using the **SetBuddy** function.

After you create the object, call the **SetFocus** function to set focus to the spin button control.

Notes

▶ **To create a Windows common control by using the dialog editor**

1. Open the application in which you want to add the common control.
2. Invoke the dialog editor.
3. Insert a dialog box by using the **Insert** command on the **Resource** menu.
4. Drag the control you want to add from the **Controls** palette onto the dialog box. This adds the specifications for the control to the dialog box resource.

The framework creates the controls when you construct a dialog object and make a call to its **Create** function or to its **DoModal** function.

Notes

131 Setting Windows Common Control Attributes

All Windows common controls have a set of attributes. The attributes for a control depend on the type of control. For example, the spin button control has a range attribute, which you can set by using the **SetRange** member function as shown in the following example:

```
m_wndCSpinbutton.SetRange(0,10);
```

The **GetRange** member function retrieves the integer values for the range set for the spin button control, as shown in the following example:

```
m_wndCSpinbutton.GetRange(nLower,nUpper);
```

The **SetBuddy** member function sets the buddy window for a spin button control that has been created by using the UDS_SETBUDDYINT style. The following example sets the buddy window of the spin button control **m_wndCSpinbutton** to the edit control **m_wndCEdit**:

```
m_wndCSpinbutton.SetBuddy(&m_wndCEdit);
```

Notes

132 Processing Notification Messages from Windows Common Controls

Applications depend on messages to determine what course of action to take. Whenever an event occurs, such as clicking a button, a message is sent from the control to its parent window. The parent window can be a frame window or a dialog box. All controls are child windows. With the exception of Windows 3.*x* controls, controls send their messages as WM_NOTIFY messages. The **OnNotify** function of the parent window handles these messages. **OnNotify** dispatches the notification message to **OnCmdMsg** for handling in message maps. The message-map entry for handling notifications is ON_NOTIFY.

The following example shows the overridden **OnNotify** function that handles the WM_NOTIFY message:

```
BOOL CSpinEditDlg::OnNotify(WPARAM wParam, LPARAM lParam, LRESULT* pResult)
{
    if(wParam==IDC_SPIN_BUTTON)
    {
        ...
        m_Pos=m_wndCSpinButton.GetPos();
        ...
    }
    return CDialog::OnNotify(wParam, lParam, pResult);
}
```

The first parameter to **OnNotify**, wParam, identifies the control that sent the message. The second parameter is a pointer to a notification structure NMHDR. The NMHDR structure contains a handle to the control sending the message, the identifier of the control, and a notification code that is specific to a control type.

Notes

133 Internet Explorer 4.0 Common Controls

Internet Explorer 4.0 common controls include the IP address control, progress control, extended combo box control, month calendar control, and the rebar control.

In this section, you will learn to create Internet Explorer 4.0 common controls, set control attributes, and process notification messages.

This section includes the following topics:

- MFC Classes for Internet Explorer 4.0 Controls
- Creating Internet Explorer 4.0 Controls
- Setting Internet Explorer 4.0 Control Attributes
- Processing Notification Messages from Internet Explorer 4.0 Common Controls

Notes

134 MFC Classes for Internet Explorer 4.0 Controls

MFC implements Internet Explorer 4.0 common controls through the classes **CIPAddressCtrl**, **CProgressCtrl**, **CComboBoxEx**, **CMonthCalCtrl**, and **CReBar**.

CIPAddressCtrl Class

The **CIPAddressCtrl** class implements the IP address control. This control is available only to applications running under Microsoft Internet Explorer 4.0 and later. **CIPAddressCtrl** is implemented in version 4.71 and later of Comctl32.dll.

The IP address control is similar to the edit control. It enables you to enter and edit an Internet Protocol (IP) address. IP addresses consist of four three-digit fields. Each field contains zero-based numbers. The IP control enables the user to enter numeric text in each field. When three digits have been entered in a given field, focus automatically moves to the next field. However, if fewer than three digits are required by the application, the user can enter the value and press the RIGHT ARROW key to move to the next field. The default range for each field is 0 to 255.To see an illustration of a sample IP Address control, click this icon.

 (CD-ROM displays the following illustration.)



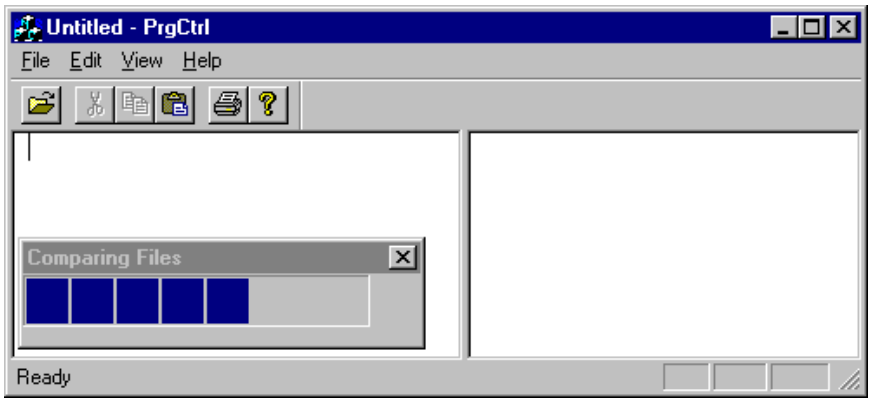
Notes

CProgressCtrl Class

The **CProgressCtrl** class implements the progress control, which is available only to applications running under Windows 95 and Windows NT version 3.51 and later.

A progress control is used to indicate the progress of a lengthy operation. It consists of a rectangle that is gradually filled, from left to right, with the system highlight color as an operation progresses. To see an illustration of a sample progress control, click this icon.


(CD-ROM displays the following illustration.)

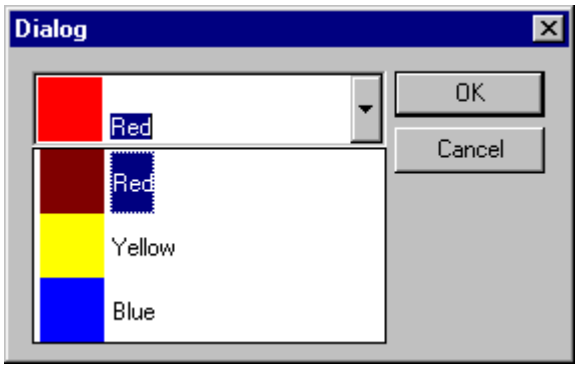


A progress control has a range, which represents the entire duration of the operation, and a current position, which represents the progress the application has made toward completing the operation. Both range and current position are represented as signed integers.

CComboBoxEx Class

The **CComboBoxEx** class extends the combo box control. It provides support for image lists. The advantage of using **CComboBoxEx** is that you do not have to implement your own image drawing code. **CComboBoxEx** can access images from an image list. To see an illustration of a sample combo box control, click this icon.


(CD-ROM displays the following illustration.)

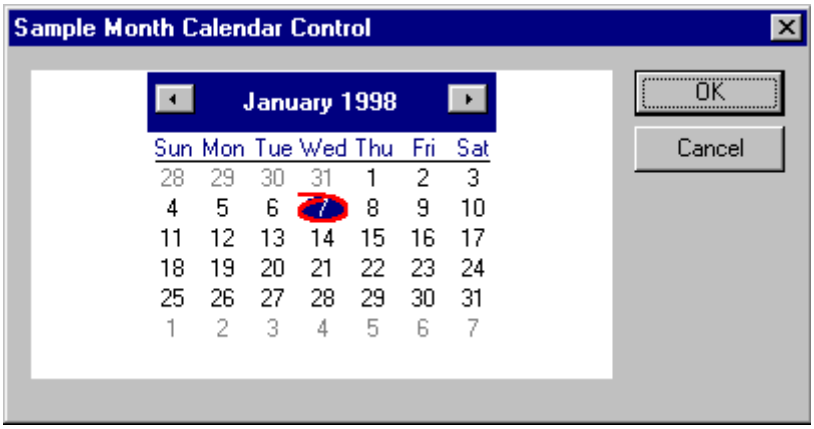


Notes

CMonthCalCtrl Class

The **CMonthCalCtrl** class implements the functionality of the month calendar control. The month calendar control provides a calendar interface from which the user can select a date. To see an illustration of a sample month calendar control, click this icon.

 (CD-ROM displays the following illustration.)



You can apply a variety of styles when you create the calendar control object. The control can display more than one month and can indicate special dates in bold.

CReBar Class

The **CReBar** class implements the rebar control. Rebars act as containers for child windows and other controls, such as toolbars and dialog bars. For information about rebars, see Using Rebars in Chapter 3: Enhancing the User Interface.

Notes

136 Creating Internet Explorer 4.0 Controls

To create any of the Internet Explorer 4.0 common controls, except the rebar control, you can use either the dialog editor or code. To create the rebar control, you must use code.

▶ **To insert an Internet Explorer 4.0 common control by using the dialog editor**

1. Open the application in which the control is to be added.
2. Create a dialog box.
3. Add the control to the dialog box.

▶ **To create an Internet Explorer 4.0 common control by using code**

1. Embed the Internet Explorer 4.0 common control object in a frame-window object or dialog box as shown in the following example:

```
class CProgressDlg : public CDialog
{
    .....
protected:
    CProgressCtrl m_wndProgCtrl;
    .....
};
```

2. Call the control's **Create** member function in the **OnCreate** function for frame windows or the **OnInitDialog** function for dialog boxes as shown in the following example:

```
BOOL CProgressDlg::OnInitDialog()
{
    CDialog::OnInitDialog();
    m_wndProgCtrl.Create(WS_CHILD|WS_VISIBLE,CRect(0,0,200,40),this,
IDC_PROGRESS);
    ....
    return TRUE;
}
```

Notes

137 Setting Internet Explorer 4.0 Control Attributes

Like Windows common controls, Internet Explorer 4.0 common controls possess attributes, which you can set by using member functions. Each control has a specific set of attributes. The class that implements the control provides functions that you invoke to set these attributes.

For example, a progress control has attributes that indicate range and a current position. The range specifies the duration of the operation, and the current position represents the progress made by the application toward completing the operation. The **CProgressCtrl** class provides functions that you invoke to set these attributes.

To initialize the progress control, call the **SetRange** and the **SetStep** functions in the **InInitDialog** function of your dialog class. The following example sets the range of the progress control **m_pProgCtrl** between 1 and 100, and sets the step increment for the progress control to one:

```
BOOL CProgressDlg::OnInitDialog()
{
    CDialog::OnInitDialog();
    m_pProgCtrl=(CProgressCtrl*)GetDlgItem(IDC_PROGRESS);
    m_pProgCtrl->SetRange(1,100);
    m_pProgCtrl->SetStep(1);
    return TRUE;
}
```

To advance the progress control by the step increment and redraw the control, call the **StepIt** function. The following example determines the range that has been set for the progress control and advances the control until the upper limit is reached:

```
int upper, lower;
m_pProgCtrl->GetRange(lower, upper);
for(; m_pProgCtrl->StepIt() < upper - 1;);
```

Notes

138 Processing Notification Messages from Internet Explorer 4.0 Common Controls

Internet Explorer 4.0 common controls send notification messages to their parent windows just as other controls do. The notification messages generated are specific to a control. The following table lists some of the notification messages generated by Internet Explorer 4.0 common controls.

Control class	Notification message	Description
CMonthCalCtrl	MCN_GETDAYSTATE	Requests information about which dates should be displayed in bold.
	MCN_SELCHANGE	Notifies the parent window that the selected date or range of the date has been changed.
	MCN_SELECT	Notifies the parent window that an explicit date selection has been made.
CComboBoxEx	CBEN_BEGINEDIT	Notifies the parent window when the user activates the drop-down list or clicks the control's edit box.
	CBEN_ENDEDIT	Sent when the user has concluded an operation within the edit box or has selected an item from the control's drop-down list.
	CBEN_INSERTITEM	Sent when a new item has been inserted in the control.
	CBEN_DELETEITEM	Sent when an item has been deleted.
CIPAddressCtrl	IPN_FIELDCHANGED	Sent when the user tabs from one IP address field to another.
CProgressCtrl	NM_OUTOFMEMORY	Sent when the control could not complete an operation because there is not enough memory available.
CReBar		Notification messages are processed for the individual controls the rebar contains.

Notes

140 ActiveX Controls

An ActiveX control is a reusable software component that supports a wide variety of functionality and can be customized to fit many software needs. An ActiveX control can be developed and reused for a variety of tasks, such as database access, data monitoring, or graphing.

This section compares ActiveX controls and common controls and explains how you can use an ActiveX control in an MFC-based application.

This section includes the following topics:

- ActiveX Controls vs. Custom Controls
- Using Automation
- ActiveX Control Containers
- ActiveX Control and Container Communication
- Implementing an ActiveX Control Container
- Using an ActiveX Control in an MFC-based Application
- Using ActiveX Controls on the Internet

Notes

141 ActiveX Controls vs. Custom Controls

Before the ActiveX technology was developed, Windows programmers designed and used custom controls when they needed to extend the set of dialog box controls.

Custom Controls

Custom controls have several limitations:

- No cross-platform support

  A custom control can be used only on and by the platform for which it was developed.
- No interaction with the container's user interface
- Not based on a standard program interface

ActiveX Controls

ActiveX controls are designed to overcome the limitations of custom controls and greatly extend a control's usability. The benefits of ActiveX controls include:

- 32-bit support
- Cross-platform capabilities

  With little or no modification, ActiveX controls can be used by environments other than the Windows operating system. This is limited by the capabilities of the target system, such as the availability and capability of the graphical interface, and the support it provides for OLE 2.0.
- Support for Automation
- Ability to send event notifications to the container
- Standardized interfaces that allow unrelated components to interconnect, much like standardized jacks on a component stereo system

Notes

142 Using Automation

Automation technology enables you to programmatically control objects that are defined outside the application that uses the objects. Automation enables programmability by exposing properties and methods of objects.

Before you implement an ActiveX control, you should understand some of the features of ActiveX controls. ActiveX controls have properties and methods that are exposed to the application in which they are contained, and can trigger events that notify the container application when an action occurs. When you develop an ActiveX control, you define the properties, methods, and events that the control will support.

To see a demonstration that shows some attributes of a sample ActiveX control named **Clock**, click this icon.

 (CD-ROM plays the demonstration, "Using ActiveX Controls.")

Properties

Properties are values that are maintained by the control and that change the appearance or behavior of an ActiveX control. You can programmatically set and retrieve properties of a control. Examples of control properties are background color and the property-enabled state.

In addition to properties exposed by a control, the ActiveX control container can expose a set of ambient properties to the control. These properties maintain the characteristics or state of the container itself, such as its background color. A control can use these properties to blend into the container's look and feel. For example, a control can set its own background color to the background color of the container.

Methods

Methods are actions that the control can perform at the request of the container. For example, a calculator control can have a method called **Calculate** that uses the current properties to compute an answer.

Events

Events are notifications that the control can send to an ActiveX control container in response to user interaction or some other occurrence. For example, when a user clicks the mouse on a control, the control sends a **Click** event notification to the container.

Type Library

For a container to use an ActiveX control, the container must know what properties and methods the control supports. The control's type library provides this information. A type library file contains prototyping information about an object's properties and methods. Using the prototyping information, ClassWizard generates a wrapper class that contains member functions for each method and property.

When you use ControlWizard to create an ActiveX control project, an .ocx file and an .odl (Object Description Language) file are created. The .ocx file is the ActiveX control and the .odl file is a text file that provides the specification for the interface that an object supports, including its properties, methods, and events. When you compile the control, the information from the .odl file is placed in a binary type-library (.tlb) file to provide the control with a common interface for many different kinds of applications.

Applications and browsers can access the type library to determine the interfaces, properties, methods, and events that a control supports.

Notes

144 ActiveX Control Containers

An ActiveX control is implemented as an in-process (DLL) server that you can use in any control container. A control container is a special type of application that supports embedding of ActiveX controls and can receive and process event notifications from the control.

For each embedded control, the control container creates an associated client site object through which the control and container interact. The control container interacts with the control through exposed methods and properties. The control interacts with the container by firing events to notify the container that an action has occurred.

The requirements of a control container are extensive and complex. MFC supplies complete control container support, which hides much of the complexity from you.

Notes

145 ActiveX Control and Container Communication

When you use an ActiveX control within a control container, two types of communication are exposed:

- Communication from the container to the ActiveX control
- Communication from the ActiveX control to the container

Communication from the Container to the ActiveX Control

The control container communicates with the ActiveX control by setting properties and invoking methods of the control. In effect, the container calls the appropriate functions in the control. An ActiveX control provides the **IDispatch** interface to handle properties and methods. **IDispatch** exposes objects, methods, and properties to a container application by providing member functions such as **Invoke**. **Invoke** provides access to properties and methods exposed by an object.

Communication from the ActiveX Control to the Container

The ActiveX control communicates with the control container by sending event notifications to the control container. A container provides a dispatch interface to handle events.

To see an animation that shows the communication flow between ActiveX controls and control containers, click this icon.

 (CD-ROM plays the animation, "Communication Between ActiveX Controls.")

Notes

147 Implementing an ActiveX Control Container

MFC provides many features that hide the complexity of creating container applications from you. However, creating a useful application that acts as a control container is a fairly complicated process. Consider a word processor, for example. Providing control container support is only one issue. You must also consider the word processor text formatting, text wrapping, merging text with the control, and user interaction with the control. These are complex issues, beyond simply allowing controls to be embedded.

You can implement a control container in three ways:

1. Use MFC AppWizard to create your application and, in Step 3, select **ActiveX Control.**
2. Manually add support by modifying your application to include Afxdisp.h in the Stdafx.h file, and update the derived **CWinApp** class **InitInstance** function by making a call to **AfxEnableControlContainer**, as shown in the following example.

```
// CContainerApp initialization
BOOL CContainerApp::InitInstance()
{
    AfxEnableControlContainer();
...
}
```

3. Use the Add ActiveX Control Containment component in the Components and Controls Gallery.

To use an ActiveX control in your control container application, you must first add the control to your application's project. In Visual C++, you use the Components and Controls Gallery to add an ActiveX control to a project. The ActiveX control is then placed in the Controls palette, where you can use it to create an instance of the control on a form.

Notes

148 Using an ActiveX Control in an MFC-based Application

To use an ActiveX control in an MFC-based application, you follow these general steps:

1. Register the control.
2. Add the control to the project.
3. Add the control to the application.
4. Access the control's methods and properties.
5. Add event handlers for the control.

These steps are described in more detail below. The steps assume that you are using an ActiveX control that was built with the MFC ActiveX ControlWizard. An ActiveX control built this way has type information embedded in the .ocx file, and the MFC class **COleControl** provides much of the control's essential functionality. To see a demonstration that shows the procedure for adding an ActiveX control to an application, click this icon.

 (CD-ROM plays the demonstration, "Adding an ActiveX Control to an Application.")

Registering the Control

If the control was built on another computer, you will need to register the control before you start Microsoft Visual Studio. Place the control in the appropriate folder, such as C:\Windows\System, and then run Regsvr32.exe and pass the control file name as a parameter to register the control.

Adding the Control to the Project

1. Start Visual Studio and load your application.

   If you will use the control directly on the application, set your application type to be a dialog-based application or an SDI or MDI application with its view class derived from **CFormView**.

   If you will use the control in one of your application's dialog boxes, it does not matter what type of application you build.

2. Locate the control in the Components and Controls Gallery and add it to the project.

   Visual Studio reads the control's type information and builds a wrapper class to expose the control's properties, methods, and events. The wrapper class is then inserted into the project in the usual manner, with an implementation and a header file.

Notes

For example, adding the **Clock** control to an MFC project adds two files, Clock.cpp and Clock.h, to the project. The **Clock** control can be found in \Samples\Ch06\Clock. To see the contents of Clock.h, click this icon

 (CD-ROM displays the following sample code.)

```
//Samples\CH06\Clock
//Clock.h - header file

class CClock : public CWnd
{
protected:
    DECLARE_DYNCREATE(CClock)
public:
    CLSID const& GetClsid()
    {
        static CLSID const clsid
            = { 0x6eb850a3, 0x263c, 0x11cf,
                { 0xa1, 0x51, 0x0, 0xaa, 0x0, 0x37, 0x4d, 0xd8 } };
        return clsid;
    }
    virtual BOOL Create(LPCTSTR lpszClassName,
        LPCTSTR lpszWindowName, DWORD dwStyle,
        const RECT& rect,
        CWnd* pParentWnd, UINT nID,
        CCreateContext* pContext = NULL)
            { return CreateControl(GetClsid(), lpszWindowName,
                        dwStyle, rect, pParentWnd, nID); }

    BOOL Create(LPCTSTR lpszWindowName, DWORD dwStyle,
        const RECT& rect, CWnd* pParentWnd, UINT nID,
        CFile* pPersist = NULL, BOOL bStorage = FALSE,
        BSTR bstrLicKey = NULL)
            { return CreateControl(GetClsid(), lpszWindowName,
                                dwStyle, rect, pParentWnd, nID,
                            pPersist, bStorage, bstrLicKey); }
```

(Code continues on following page.)

Notes

```
// Attributes

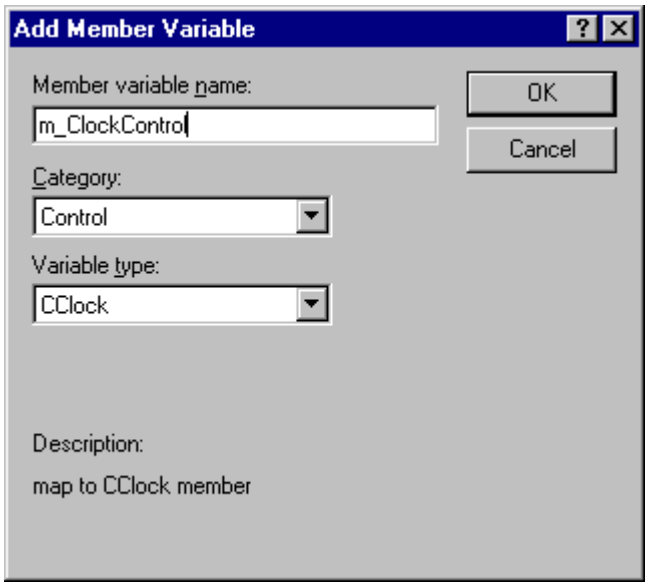
public:
    short GetAlarmHour();
    void SetAlarmHour(short);
    short GetAlarmMinute();
    void SetAlarmMinute(short);
    short GetAlarmType();
    void SetAlarmType(short);
    CString GetAlarmSound();
    void SetAlarmSound(LPCTSTR);
    CString GetAlarmCommand();
    void SetAlarmCommand(LPCTSTR);
    BOOL GetAlarmSet();
    void SetAlarmSet(BOOL);
    BOOL Get_AlarmSet();
    void Set_AlarmSet(BOOL);
    OLE_COLOR GetBackColor();
    void SetBackColor(OLE_COLOR);

// Operations
public:
    void SetAlarmTime(short hour, short nNewValue);
    void TestAlarm();
    void AboutBox();
};
```

Notes

Adding the Control to the Application

You can use an ActiveX control directly on the application or in the application's dialog box. If you add a control to a dialog box, you can size the control and set design-time properties as needed. Use ClassWizard to add a variable to the class that contains the dialog box. For instance, in an SDI application whose view class is derived from **CFormView**, add a member variable to the application's view class. The following illustration shows the addition of a member variable of type **CClock**.



Accessing the Control's Methods and Properties

Once you have added the control to your application, you can write code to access the control's methods and properties.

The following example code sets the **AlarmHour** property of the **Clock** control:

```
m_ClockControl.SetAlarmHour(10);
```

The following example code retrieves the **AlarmHour** property of the **Clock** control:

```
short hours = m_ClockControl.GetAlarmHour();
```

To set a parameterized property, controls typically expose a method that accepts multiple arguments. For example, the **Clock** control provides the **SetAlarmTime** method, which takes two arguments. This method sets the parameterized property **AlarmTime**. The following example invokes the **SetAlarmTime** method:

```
m_ClockControl.SetAlarmTime(10, 23);
```

The control's methods, like its properties, are member functions of the wrapper class that Visual Studio created from the control's type library. The following example tests the alarm and sets the command to be executed when the alarm triggers:

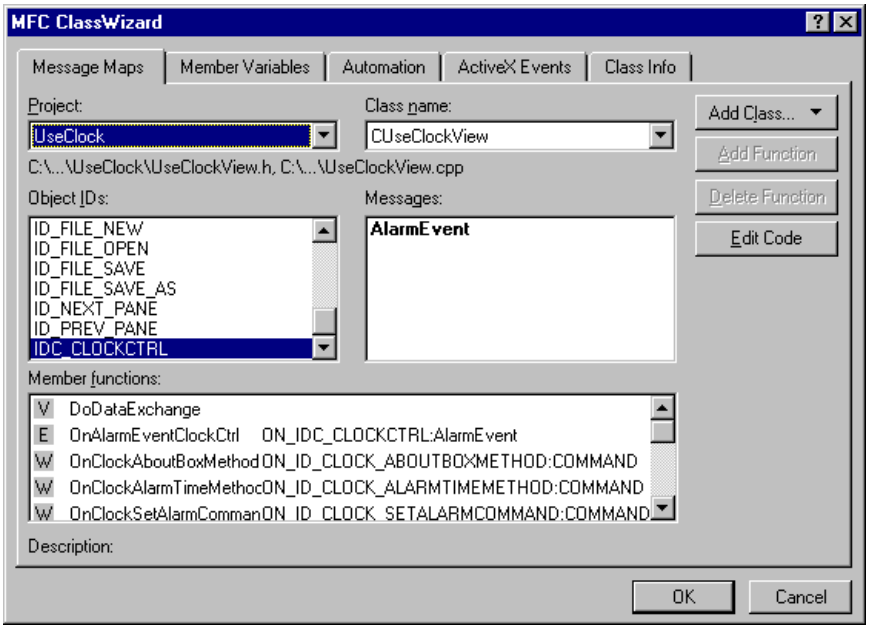
```
m_ClockControl.TestAlarm();
m_ClockControl.SetAlarmCommand("calc.exe");
```

Notes

Adding Event Handlers for the Control

Use ClassWizard to add event handlers for events the control fires. On the **Message Maps** tab in the **ClassWizard** dialog box, set the **Class name** list box to the class that contains the ActiveX control. Set the **Object IDs** list box to the ActiveX control. The events associated with the control will be displayed in the **Messages** dialog box. Add a function to receive that event, just as you would add a command handler. To see an illustration of the **Messages** dialog box, click this icon.

 (CD-ROM displays the following illustration.)



ClassWizard handles events differently than messages. ClassWizard inserts an event sink map in the class, with an entry for each event handler you install using ClassWizard, as shown in the following example:

```
BEGIN_EVENTSINK_MAP(CUseClockView, CFormView)
    ON_EVENT(CUseClockView, IDC_CLOCKCTRL, 1,
            OnAlarmEventClockCtrl, VTS_NONE)
END_EVENTSINK_MAP()
```

Write the event handler code as needed for the control:

```
void CUseClockView::OnAlarmEventClockCtrl()
{
    MessageBeep(-1);
    MessageBox("The control has fired its alarm event!");
}
```

For a complete application that uses the **Clock** control and shows how to access all its methods, properties, and events, see the sample application UseClock, which is located in the folder Samples\Ch06\UseClock.

Notes

150 Using ActiveX Controls on the Internet

Many of the ActiveX controls that you create and use in MFC can be used on Web pages. You can use Microsoft FrontPage 98® to create Web applications with controls embedded in them.

Performance Issues

When selecting or creating ActiveX controls for use on the Internet, you must consider whether the control is suitable for the environment. The size of an ActiveX control dramatically affects its usability and performance on a Web page. Consider the size of the sample MFC-based **Clock** control found in \Samples\Ch06\Clock. A release version of the **Clock** control is about 31 KB. There are two DLLs required of all MFC-based applications or components: Mfc42.dll (918 KB) and Msvcrt.dll (256 KB). Therefore, the size of the **Clock** control and its auxiliary files is approximately 1205 KB—a large application to download from a Web page.

Note The sizes specified here are uncompressed. Microsoft CAB compression typically reduces file sizes by half.

Fortunately, the files that need to be downloaded for a specific control usually are not this large. The MFC DLLs will already be present on any computer using Internet Explorer 3.0 or later. If they are not present because the user has another browser installed, then your Web page must download the MFC DLLs. For information about how to do this, search for "Internet First Steps: Upgrading an Existing ActiveX Control" in MSDN Library Visual Studio 6.0.

Modifying ActiveX Controls for the Internet

When you use an ActiveX control on a Web page, it should download quickly and be as small as possible. For information about how to make your control small and fast, search for "ActiveX Controls: Optimization" in MSDN Library Visual Studio 6.0.

If your control has large amounts of persistent data, such as images or video data, you can improve its downloading times by downloading the data asynchronously. You can add this feature to your control in Step 2 of the MFC ActiveX ControlWizard by clicking the **Advanced** button and choosing **Loads Properties Asynchronously**. The manual process for adding asynchronous downloading to a control's properties is complex and is beyond the scope of this course.

Using the Active Template Library (ATL)

You can also use the ATL COM AppWizard to build ActiveX controls. This method requires more manual programming than creating controls using ControlWizard. However, you can build a much smaller, faster control that has only the components it needs. For information about building ActiveX controls using the ATL COM AppWizard, search for "Using the ATL COM AppWizard" in MSDN Library Visual Studio 6.0.

Notes

151 Controls Supplied by MFC

Apart from Windows common controls, Internet Explorer 4.0 common controls, and ActiveX controls, MFC provides the following three controls that you can use in your applications:

- **CBitmapButton**

  Bitmap buttons are controls that are labeled with bitmapped images instead of text.

- **CCheckListBox**

  Check list boxes are similar to list boxes, but each item in the list has a check box next to it, which can be selected or cleared.

- **CDragListBox**

  Drag list boxes are similar to list boxes, with the additional capability of allowing the user to order the items in the list by dragging them to change their positions.

This section describes how to create and handle notification messages from a bitmap button.

This section includes the following topics:

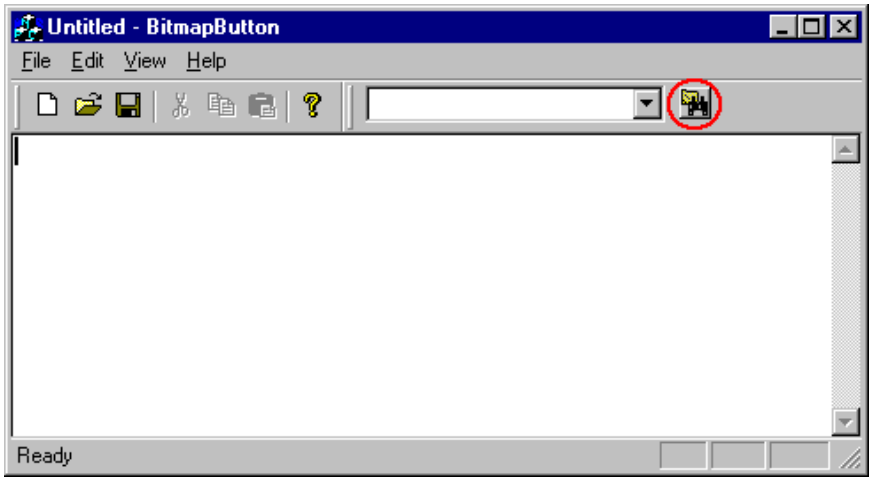
- Adding a Bitmap Button to an Application
- Processing Notification Messages from Controls Supplied by MFC

Notes

152 Adding a Bitmap Button to an Application

The **CBitmapButton** class implements the bitmap button. Objects created from the **CBitmapButton** class can contain four images, which correspond to the four states of the button: up, down, focused, and disabled. Only the bitmap for the up state is required. To see an illustration of a bitmap button on a dialog bar, click this icon.

(CD-ROM displays the following illustration.)



You can create **CBitmapButton** controls by calling either the **Create** function or the **AutoLoad** function.

▶ **To add a bitmap button control by using Create**

1. Create a bitmap image for the button. You can create additional images for the button, in case you want to display a different image for each state.
2. Declare a **CBitmapButton** object in the mainframe class.
3. In the **OnCreate** function of the mainframe class, call the **Create** function to create the bitmap control and attach it to the **CBitmapButton** object as shown in the following example:

```
m_wndBitmapButton.Create("",BS_OWNERDRAW|WS_VISIBLE,
                        CRect(200,0,232,32),&m_wndDlgBar,ID_FINDTEXT);
```

4. After the bitmap button is constructed, call the **LoadBitmaps** member function to load the bitmap resources as shown in the following example:

```
m_wndBitmapButton.LoadBitmaps(IDB_FINDU);
```

▶ **To add a bitmap button control in a dialog bar by using AutoLoad**

1. In the mainframe class, declare a **CBitmapButton** object.
2. Using the dialog editor, create a dialog bar and add a command button to it. Position the command button where you want the bitmap button to appear. Set the command button's style to owner draw by using the properties dialog box.
3. Set the button's caption (for example, IDB_RED).

Notes

4. Create a bitmap image for the button. You can create additional images for the button, in case you want to display a different image for each state.
5. In the properties dialog box for each image, assign each image an ID.

   The ID for each image which represents a state of the bitmap button, is assigned by taking the caption of the command button created in step 2, and appending one of the letters “U,” “D,” “F,” or “X” to it to indicate the state that the image represents.

   The following table lists the letters to be appended while assigning IDs to the images that represent the states of a bitmap button:

Letter	State	Example
U	Up	"IDB_REDU"
D	Down	"IDB_REDD"
F	Focused	"IDB_REDF"
X	Disabled	"IDB_REDX"

   When assigning IDs to the images, make sure that the IDs are within double quotes; otherwise, the image will fail to load.
6. Call the **CBitmapButton** object’s **AutoLoad** function, using as parameters the button’s control ID and the address of the dialog bar as shown in the following example:

   ```
   m_wndBitmapButton1.AutoLoad(ID_COLORS_RED,&m_wndDlgBar);
   ```

   In this example, ID_COLORS_RED is the name of the button.

Notes

155 Processing Notification Messages from Controls Supplied by MFC

Bitmap buttons generate the same notification messages as control buttons do. Possible message-map entries include ON_BN_CLICKED and ON_BN_DOUBLECLICKED.

▶ **To handle notification messages from a bitmap button on a dialog bar**

1. Add derived object message-map entries to the class in which the bitmap-button messages are to be handled. This could be the mainframe class or the view class as shown in the following example:

```
BEGIN_MESSAGE_MAP(CBitmapButtonView, CEditView)
    //{{AFX_MSG_MAP(CBitmapButtonView)
    ....
    ON_BN_CLICKED(ID_FINDTEXT,OnBnClicked)
END_MESSAGE_MAP()
```

2. Add message-handler functions to the class in which the bitmap-button messages are being handled.

Notification messages from a bitmap-button on a dialog bar can also be handled in the view, as shown in the following example:

```
void CBitmapButtonView::OnBnClicked()
{
    if(!m_Text.IsEmpty())
    {
        OnFindNext(m_Text,TRUE,TRUE);
    }
}
```

The function **OnBnClicked** is invoked in response to the message ON_BN_CLICKED. You must add the following message-map entry in the message map of the view class:

```
ON_BN_CLICKED(ID_FINDTEXT,OnBnClicked)
```

The function invokes the **OnFindNext** function to search the view for the text contained in m_Text. If m_Text is empty, no search occurs.

Note If the bitmap button is part of a dialog box, add the message-map entry and the message-handler to the dialog object.

Notes

157 Sample Applications

The following table describes the sample applications related to this chapter. These sample applications are located in the folder \Samples\Ch05.

Sample application subfolder	Description of application
BitmapBut	Uses the bitmap-button control to change the color of text displayed.
ComboBoxEx	Uses an extended **ComboBox** control to change the color of text displayed.
ProgressCtrl1	Displays the extent of progress of comparing two files in a dialog box. The progress control is created using the dialog editor.
ProgressCtrl2	Displays the extent of progress in comparing two files in a dialog box. The progress control is created using code rather than the dialog editor.
Spinbutton	Displays the product of two numbers, one from a spin control and the other from an edit control.

Notes

161 Self-Check Questions

1. What are events?

- A. The actions that the ActiveX control can perform at the request of the container
- B. The settings that change the appearance or behavior of an ActiveX control
- C. The notifications that an ActiveX control container sends to an ActiveX control in response to user interaction or some other occurrence
- D. The notifications that an ActiveX control sends to an ActiveX control container in response to user interaction or some other occurrence

2. What does a .odl file contain?

- A. The specification for the interface that an object supports, including its properties, methods, and events, in text format
- B. Information to provide a common interface for the control with many different kinds of applications, in binary format
- C. Executable code for the ActiveX control
- D. Dynamic-link library routines that are loaded with the ActiveX control

3. Which notification message is sent if the progress control cannot complete an operation because there is not enough memory available?

- A. MCN_GETDAYSTATE
- B. IPN_FIELDCHANGED
- C. NM_OUTOFMEMORY
- D. CMemoryException

Notes